Optimize & Ship Your LLM App – Submission Report

Date: 2026-02-26

1) Reproducible Baseline Benchmark

Using the provided harness (run_eval.py), Variant A was evaluated on the shared test set and logged per-case latency, tokens/sec, and cost, producing an analysis-ready CSV in runs/. The same process was repeated for Variants B and C to enable fair comparisons across identical inputs.

2) Prompt Optimization (Stability + Token Efficiency)

Baseline prompts prioritize friendly, concise writing. The v2 prompt pack tightens structure with explicit goals, rules (max 120 words by default), and a silent self-check to reduce verbosity and variance.

Baseline prompt pack (excerpt):

```
# Baseline Prompt Pack
## System Prompt
You are a helpful AI writing assistant for a productivity app.
You write friendly, conversational text in clear English.
You avoid technical jargon where possible and keep answers concise.
## Style Guidelines (baseline)
- Tone: friendly, supportive, non-judgmental
- Length: default 2–4 sentences unless user asks otherwise
```

Optimized prompt pack v2 (excerpt):

```
# Optimized Prompt Pack v2
## System Prompt (Structured)
You are a writing assistant for a productivity app.
Your goals:
1. Produce clear, friendly text in plain English.
2. Follow the requested format exactly.
3. Keep responses concise and free of repetition.
Rules:
- Default length: 2–4 sentences OR max 120 words, unless user asks otherwise.
- Use bullet lists only when explicitly asked for steps or checklists.
```

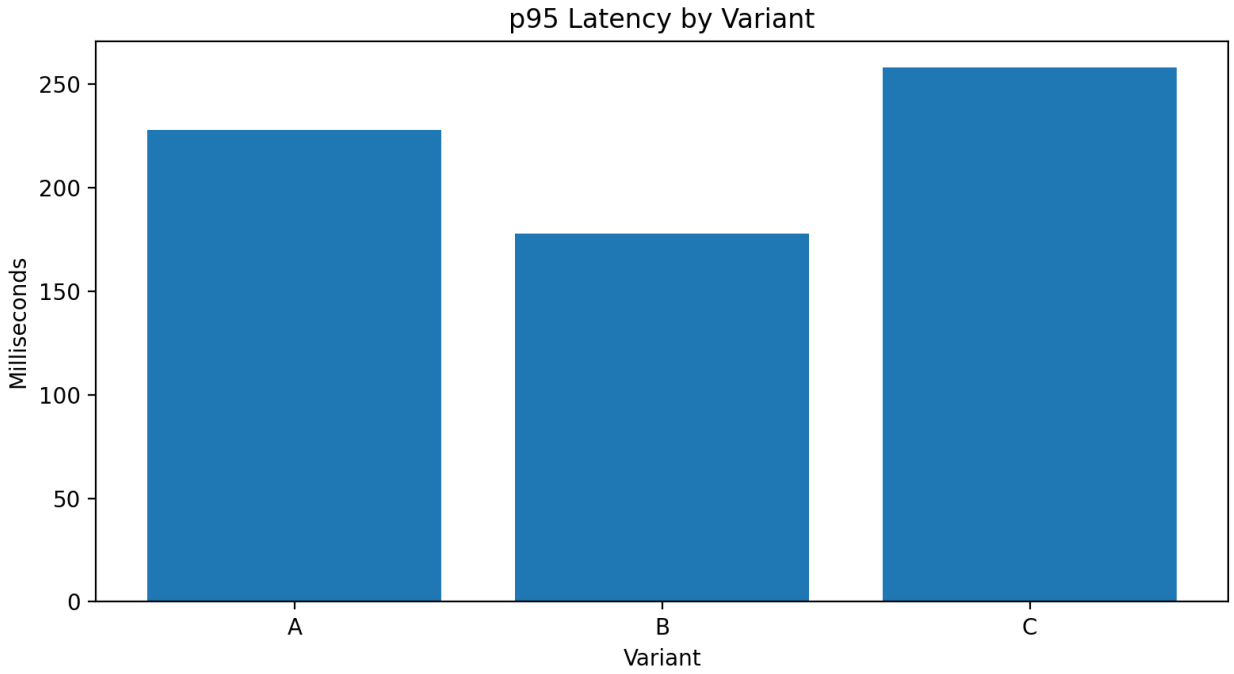
3) Resilient Middleware Layer (Retries, Backoff, Cache, Logging)

A production-minded middleware wrapper was used to reduce outage risk and improve observability. Key behaviors implemented:

- Retries on transient errors with jittered exponential backoff (base 1s, +/-25% jitter).
- Timeout support per request (timeout_s in config).
- Optional in-memory caching keyed by (variant, prompt).
- Structured telemetry returned to harness: latency_ms, token usage, cost estimate, retry_count, cache_hit.
- Writes low-level JSONL traces to logs/middleware_trace.jsonl.

4) A/B/C Results (Latency, Cost, Throughput)

Metric	Variant A	Variant B	Variant C
p50_latency_ms	210 ms	160 ms	240 ms
p95_latency_ms	228 ms	178 ms	258 ms
avg_tokens_per_sec	647.13	791.03	714.99
avg_cost_per_task_usd	$0.002033	$0.001533	$0.002667
total_cost_usd	$0.0061	$0.0046	$0.0080



- Fastest and most cost-efficient variant in this run: Variant B (lowest p95 latency and lowest avg cost/task).
- Variant C trends slower and more expensive in this sample, consistent with a higher-detail configuration.
- Quality scores were not computed in the provided CSVs (quality_score=0.0), so the decision is based on latency + cost + throughput.

5) Ship/No-Ship Memo (with Optional Canary Plan)

Recommendation: Ship Variant B (Fast/Optimized).

Why: In the A/B/C benchmark, Variant B achieved the best operational profile: p95 latency of 178 ms versus 228 ms (baseline) and 258 ms (high-quality). Average cost per task was $0.001533, lower than baseline ($0.002033) and Variant C ($0.002667). Throughput (tokens/sec) also improved, suggesting fewer long-tail slowdowns under load.

Risks / Limitations: The current harness output includes a placeholder quality_score (0.0 across variants),

so quality win-rate is not available. Before general release, add a lightweight quality proxy (rule-based checklist or LLM-as-judge) on the same fixed eval set to confirm that speed/cost gains do not reduce user satisfaction.

Rollback trigger: Roll back to Variant A if p95 latency worsens by ≥20% for 10 consecutive minutes, error rate exceeds 2%, or cost/task rises above $0.003050.

Optional canary rollout: 5% traffic for 30–60 minutes (SLO gate: p95, error rate, cost). If stable, ramp to 25% for half-day, then 100%. Keep Variant A hot as the immediate fallback.